

The basic code for building gin app:

```
package main

import "github.com/gin-gonic/gin"

func main() {
    router := gin.Default();

    router.GET("/", func (c *gin.Context) {
        c.JSON(200, gin.H{
            // the comma at the end is required
            "msg": "Hello Jafar Loka World",
        })
    })
    // the default port is: 8080
    // router.Run()
    router.Run(":5000")
}
```

To list all modules for project: *go list -m all*

To remove any unused module: *go mod tidy*

To refactor code:

```
package main

import "github.com/gin-gonic/gin"

func HelloWorldHandler(c *gin.Context) {
    c.JSON(200, gin.H{
        "msg": "Hello Jafar Loka World",
    })
}

func main() {
    router := gin.Default()

    router.GET("/", HelloWorldHandler)

    router.Run(":5000")
}

*****
```

To build types into for xml response:

```
import (
    "encoding/xml"

    "github.com/gin-gonic/gin"
)

type Person struct {
    XMLName xml.Name `xml:"PersonObject"`
    // here the data and attr must start with capital letter
    FullName string `xml:"FullName"`
    Age      int16  `xml:"Age,attr"`
}

*****
```

To build path param and xml response in gin:

```
package main

import (
    "encoding/xml"
    "github.com/gin-gonic/gin"
)

type Person struct {
    XMLName xml.Name `xml:"PersonObject"`
    // here the data and attr must start with capital letter
    FullName string `xml:"FullName"`
    Age      int16  `xml:"Age,attr"`
}

func HelloWorldHandler(c *gin.Context) {
    c.JSON(200, gin.H{
        "msg": "Hello Jafar Loka World",
    })
}

func HelloWorldNameHandler(c *gin.Context) {
    name := c.Params.ByName("name")
    c.JSON(200, gin.H{
        "msg": "Hello To JLoka World: " + name,
    })
}

func FullNameHandler(c *gin.Context) {
    c.XML(200, Person{FullName: "Jafar Loka-01", Age: 25})
}

func main() {
    router := gin.Default()

    router.GET("/", HelloWorldHandler)

    router.GET("/:name", HelloWorldNameHandler)

    router.GET("/fullName", FullNameHandler)

    router.Run(":5000")
}
```

For json data model:

```
import (  
    "net/http"  
    "time"  
  
    "github.com/gin-gonic/gin"  
    "github.com/rs/xid"  
)  
  
type Meal struct {  
    ID          string    `json:"id"`  
    Name        string    `json:"name"`  
    Tags        []string  `json:"tags"`  
    Ingredients []string  `json:"ingredients"`  
    Instructions []string  `json:"instructions"`  
    CreatedAt   time.Time `json:"createdAt"`  
}
```

```

var meals []Meal

func CreateMealHandler(c *gin.Context) {
    var meal Meal

    if err := c.ShouldBindJSON(&meal); err != nil {
        c.JSON(http.StatusBadRequest, gin.H{
            "err": err.Error(),
        })
    }
    meal.ID = xid.New().String()
    meal.CreatedAt = time.Now()
    meals = append(meals, meal)
    c.JSON(http.StatusCreated, meal)
}

func GetAllMealsHandler(c *gin.Context) {

    if len(meals) == 0 {
        c.JSON(http.StatusOK, gin.H{
            "msg": "No Data Found",
        })
        return
    }
    c.JSON(http.StatusOK, meals)
}

func main() {
    router := gin.Default()

    meals = make([]Meal, 0)

    router.POST("/meals", CreateMealHandler)

    router.GET("/meals", GetAllMealsHandler)

    router.Run()
}

*****

```

```

func main() {
    router := gin.Default()
    meals = make([]Meal, 0)

    router.POST("/meals", CreateMealHandler)
    router.GET("/meals", GetAllMealsHandler)
    router.PUT("/meals/:id", UpdateMealHandler)
    router.DELETE("/meals/:id", DeleteMealHandler)
    router.GET("/meals/searchByTag", SearchForMealByTag)

    router.Run()
}

```

```

func UpdateMealHandler(c *gin.Context) {
    id := c.Param("id")

    var meal Meal

    if err := c.ShouldBindJSON(&meal); err != nil {
        c.JSON(http.StatusBadRequest, gin.H{
            "err": err.Error(),
        })
        return
    }

    for i := 0; i < len(meals); i++ {
        if meals[i].ID == id {
            meal.CreatedAt = meals[i].CreatedAt
            meal.ID = id

            meals[i] = meal
            c.JSON(http.StatusOK, meal)
            return
        }
    }

    c.JSON(http.StatusNotFound, gin.H{
        "err": "No Meal Found",
    })
}

```

```

func DeleteMealHandler(c *gin.Context) {
    id := c.Param("id")
    for i := 0; i < len(meals); i++ {
        if meals[i].ID == id {
            meals = append(meals[:i], meals[i+1:]...)
            c.JSON(http.StatusNoContent, gin.H{
                "msg": "Meal Deleted successfully",
            })
            return
        }
    }
    c.JSON(http.StatusNotFound, gin.H{
        "err": "No Meal Found",
    })
}

func SearchForMealByTag(c *gin.Context) {
    tag := c.Query("tag")
    t1 := make([]Meal, 0)

    for i := 0; i < len(meals); i++ {
        t2 := false
        for _, t := range meals[i].Tags {
            if strings.EqualFold(t, tag) {
                t2 = true
                break
            }
        }

        if t2 {
            t1 = append(t1, meals[i])
        }
    }

    c.JSON(http.StatusOK, t1)
}

*****

```

For documenting the app:

```
// Package main My Simple Meal API
//
// The purpose of this application is to demonstrate go-swagger
//
// Schemes: http
// Host: localhost:8080
// BasePath: /
// Version: 1.0.0
//
// Consumes:
// - application/json
//
// Produces:
// - application/json
//
// swagger:meta
```

And for documenting the models:

```
// Meal represents a user in the system.
//
// swagger:model Meal
type Meal struct {
    // ID of the Meal
    // required: true
    ID string `json:"id"`
    // Name of the Meal
    // required: true
    Name string `json:"name"`
    // Tags of the meal
    // required: true
    Tags []string `json:"tags"`
    // Ingredients of meal
    // required: true
    Ingredients []string `json:"ingredients"`
    // Instructions to create meal
    // required: true
    Instructions []string `json:"instructions"`
    // When we create the record
    // required: true
    CreatedAt time.Time `json:"createdAt"`
}
```

And for documenting the GET Methods without any params or query data:

```
// swagger:operation GET /meals meals listOfMeals
//
// Returns list of meals
// ---
//
// produces:
// - application/json
// responses:
//   '200':
//     description: list of meals
```

And for documenting params:

```
// swagger:operation PUT /meals/{id} meals UpdateMealById
// Update meal by id if exists only
// ---
//
// parameters:
// - name: id
//   in: path
//   description: id of the meal
//   required: true
//   type: string
// produces:
// - application/json
// responses:
//   '200':
//     description: Meal Updated Successfully
//   '400':
//     description: Bad Request
//   '404':
//     description: Meal not found
```

And for make the `swagger.json` or `swagger.yaml` and for models:

Run command: `go-swagger generate spec -o ./swagger.json --scan-models`

And to run the swagger:

Run command: `go-swagger serve -F swagger ./swagger.json`

For create meal:

```
// swagger:operation POST /meals meals CreateMealHandler
// Create meal
// ---
//
// parameters:
//   - name: meal
//     in: body
//     description: The body of request
//     required: true
//     schema:
//       "$ref": "#/definitions/meal"
// consumes:
//   - application/json
// produces:
//   - application/json
// responses:
//   '201':
//     description: Meal Created Successfully
//     schema:
//       "$ref": "#/definitions/meal"
//   '400':
//     description: Bad Request
```

For search by tag in query:

```
// swagger:operation GET /meals/searchByTag meals SearchForMealByTag
//
// Returns list of meals that contains the tag
// ---
// parameters:
//   - name: tag
//     in: query
//     description: tag of the meal
//     required: true
//     type: string
//
// produces:
//   - application/json
// responses:
//
//   '200':
//     description: list of meals that contains the tag
```